

Topology-Aware Packet Classification and Adaptive Local-Core Selection for SpiNNaker: Design and Reproducible Virtual-Mode Benchmarking

Christopher Lee Burgess
BGecko Research
bgeckoresearch@gmail.com

June 2026

Abstract

SpiNNaker uses packet-switched multicast communication over a low-degree inter-chip topology, so routing quality depends strongly on spatial traffic structure. This paper develops a proposal for topology-aware packet classification and adaptive local-core selection, contributing in three ways. First, it clarifies the architectural split between inter-chip Router forwarding and intra-chip System Network-on-Chip (SNoC) delivery. Second, it formalizes the refinement rule as a constrained dispatch layer that can only restrict a standard base route and processor mask, thereby preserving backward compatibility through a null dispatch class. Third, it introduces a reproducible virtual-mode audit implemented on a local SpiNNaker software stack using SpiNNakerGraphFrontEnd, PACMAN, and SpiNNMachine. The audit compares a precisely defined static-routing baseline against multiple ablations across seeds, topology sizes, traffic densities, destination structures, local-mask sizes, and routing-pressure settings. In this software benchmark, the combined refinement reduced a synthetic energy proxy by 18.28% on average, with substantial variance and occasional regressions. The audit further indicates that processor-mask refinement is the dominant contributor to the proxy reduction, while space-bit direction refinement alone has smaller and less stable effects. These results are not hardware-measured energy claims; they are reproducible virtual-mode evidence that the proposal is experimentally testable and that its apparent gains depend materially on modeling assumptions.

1 Introduction

SpiNNaker is designed around sparse, event-driven multicast communication. That design makes routing efficiency important: when packets must be copied across multiple chips and delivered to selected local cores, spatial organization becomes a primary concern. In standard SpiNNaker usage, the routing key selects a routing-table entry whose link set and processor mask are effectively static during the fast path. This is efficient, but it provides limited support for per-packet specialization of forwarding geometry and local-core fan-out.

The central idea studied here is to associate each packet with a compact *dispatch class*. The standard routing key still determines the base route. The dispatch class then refines that base route by (i) restricting the permitted inter-chip forwarding directions and (ii) refining the processor-mask fan-out used for local delivery. The first refinement is router-facing. The second is SNoC-facing. This separation matters because the Router determines routing and local-delivery eligibility, whereas the SNoC carries locally delivered packets to the subset of cores selected by the effective processor mask.

This work develops both the architectural formalization and the experimental grounding together. All benchmarks reported here were obtained on a virtual SpiNNaker configuration rather than physical hardware. Accordingly, the measured quantities are software-defined proxies for energy, latency, and congestion exposure rather than board-level power measurements. Their value is methodological: they show that the proposal can be instantiated, tested, falsified, and refined within the existing SpiNNaker software ecosystem before hardware deployment.

Main contributions. This paper makes four concrete contributions:

- (C1) It formalizes a two-stage routing rule in which a standard routing lookup selects a base route and a compact dispatch class refines both permitted inter-chip directions and local processor-mask fan-out.
- (C2) It makes the Router/SNoC division of responsibility explicit, preventing the common conceptual error of attributing local-core selection directly to inter-chip routing logic.
- (C3) It develops a reproducible virtual-mode benchmark harness on top of SpiNNakerGraphFrontEnd, PACMAN, and SpiNNMachine and compares dispatch-class refinements against a standard static-routing baseline.
- (C4) It reports the experimental results conservatively: the observed gains are software-benchmark results under virtual mode, not hardware-validated power measurements.

2 Background and Architectural Model

2.1 SpiNNaker routing context

A SpiNNaker route word can encode both inter-chip forwarding and local processor delivery. This is one of the platform’s defining strengths: the same packet can be replicated across selected links and delivered locally to multiple processing cores. That same compactness also motivates the present refinement. If different packet classes require different spatial behavior or different local-core subsets, the standard approach generally allocates distinct routing keys or distinct route entries. That is workable, but it makes packet-level adaptation comparatively coarse.

2.2 Architectural split: Router versus SNoC

The proposal in this paper depends on a precise division of labor:

- **Router:** performs routing-table lookup on the packet key, determines the base inter-chip forwarding set, and determines whether local delivery is enabled.
- **SNoC:** carries the locally delivered packet to the subset of local cores selected by the effective processor mask.

The Router therefore does not itself schedule cores. Rather, it determines which cores are eligible for local delivery. The SNoC realizes that local multicast fan-out.

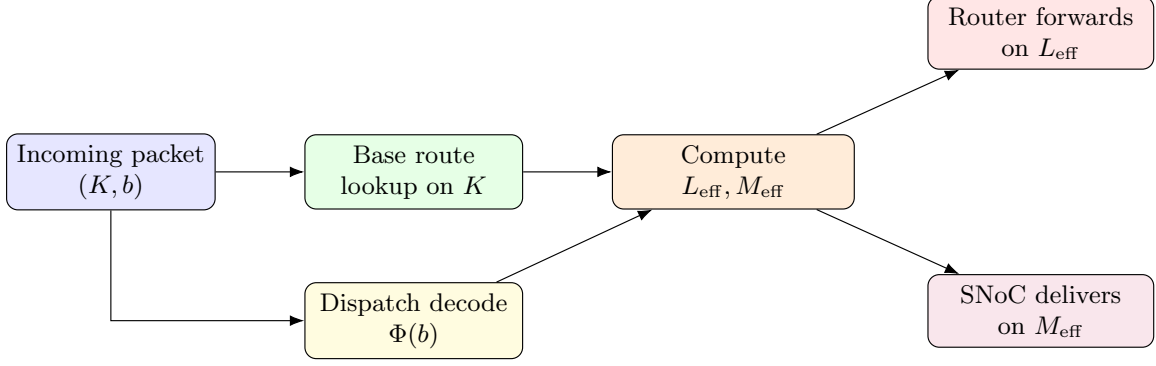


Figure 1: Two-stage routing refinement. The packet key K selects a base route; the dispatch class b selects a refinement policy. The Router applies the effective inter-chip direction set, while the SNoC applies the effective processor mask for local delivery.

3 Formal Routing Model

Let $K \in \{0, 1\}^{32}$ denote the standard SpiNNaker routing key, and let $b \in \{0, 1\}^k$ denote a compact dispatch-class field associated with the packet. A standard routing lookup on K returns a base route

$$r_{\text{base}} = (L_{\text{base}}, M_{\text{base}}),$$

where $L_{\text{base}} \subseteq D$ is the asserted inter-chip forwarding set and $M_{\text{base}} \in \{0, 1\}^{18}$ is the asserted local processor mask.

The dispatch map is

$$\Phi : \{0, 1\}^k \rightarrow \mathcal{P}(D) \times \{0, 1\}^{18}, \quad \Phi(b) = (L_b, M_b),$$

with D the set of available primitive link directions.

The effective route is then

$$L_{\text{eff}} = L_{\text{base}} \cap L_b, \tag{1}$$

$$M_{\text{eff}} = M_{\text{base}} \wedge M_b. \tag{2}$$

This formulation is intentionally conservative. The refinement mechanism can only restrict the base route; it cannot create new inter-chip directions or new processor-mask bits that were absent in the original route entry.

Proposition 1. *Let b_0 be a null-compatible dispatch class satisfying $L_{b_0} = D$ and $M_{b_0} = \mathbf{1}_{18}$. Then the refined route exactly recovers the standard SpiNNaker route:*

$$L_{\text{eff}} = L_{\text{base}}, \quad M_{\text{eff}} = M_{\text{base}}.$$

The proof is immediate from set intersection with D and bitwise conjunction with the all-ones vector.

3.1 Routing and refinement procedure

Algorithm 1 summarizes the implemented routing/refinement logic at the level appropriate for the manuscript.

Table 1: Pseudocode for topology-aware route refinement.

Input: routing key K , dispatch class b , source chip s , destination chip t , congestion set H
Output: effective path P , effective processor-mask size m_{eff}

1. Look up the base route for K and derive the baseline preferred direction order.
 2. If using the baseline or null-dispatch condition, keep the full baseline direction order and set $m_{\text{eff}} = 18$.
 3. If using Router-side refinement, derive a dispatch-dependent restricted direction order and attempt a path that avoids congested chips in H .
 4. If no valid refined path is found, fall back to the baseline direction order; if necessary, fall back again to a congestion-agnostic route.
 5. If using SNoC-side processor-mask refinement, replace the full local mask with the dispatch-selected mask size target; otherwise keep $m_{\text{eff}} = 18$.
 6. Compute benchmark metrics from the resulting path length, congested-hop count, and effective mask size.
-

4 Experimental Methodology

4.1 Audit setup

The audited implementation was executed locally using:

- a Python virtual environment,
- `SpiNNakerGraphFrontEnd 1!7.4.1`,
- `SpiNNaker_PACMAN 1!7.4.1`,
- `SpiNNMachine 1!7.4.1`, and
- virtual SpiNNaker topologies of size 8×8 and 12×12 .

The audit used 10 random seeds, 3 traffic-density levels, 2 destination-structure regimes, 4 local processor-mask targets, and 2 routing-pressure settings, for a total of 960 comparisons per ablation.

Baseline definition. The baseline is standard static routing under standard routing keys only, with no dispatch-class refinement and a fixed 18-core local processor mask for local delivery. In the audit model, the baseline route is determined solely by the base-route logic.

Proposed method definition. The proposed method is represented in software through dispatch-class policies. Dispatch classes may (i) alter preferred inter-chip forwarding order and congestion-avoidance behavior and/or (ii) reduce the effective local processor-mask size. Inter-chip direction refinement is modeled as Router-side policy, while processor-mask refinement is modeled as SNoC-side local-delivery policy.

Implementation summary. The baseline was simulated as shortest-path static routing under standard routing keys only, with no dispatch refinement and a fixed 18-core local processor mask. The proposed method used the same workloads, but introduced dispatch-class logic in two optional places: a Router-side policy that changes direction preference and congestion avoidance, and an

SNoC-side policy that changes the effective local processor-mask size. These mechanisms were activated independently in the ablation study so that their individual effects could be measured separately.

Energy and latency metric audit. The reported *energy proxy* is defined by

$$E_{\text{proxy}} = \text{hops} + 0.18 \text{ mask_size} + 0.9 \text{ congested_hops}.$$

The *latency proxy* is defined by

$$L_{\text{proxy}} = \text{hops} + 1.75 \text{ congested_hops}.$$

These are **synthetic simulation proxies**, not hardware-calibrated models. They do not measure board-level energy, router micro-activity, SDRAM traffic, monitor-core overhead, or physical SNoC timing. The energy proxy rewards reduced processor-mask size by construction, so any energy-savings interpretation must be stated conservatively.

Fairness constraints. Baseline and proposed variants were run on identical workloads. Source/destination pairs, congestion hot spots, topology size, traffic density, mask target, and routing pressure were held fixed within each comparison. No extra route-table resources were assigned to the proposed method. The null-dispatch case was required to recover the baseline exactly, and all ablations were evaluated under the same seed schedule and the same benchmark driver so that non-routing factors remained fixed unless intentionally varied.

4.2 Ablations

Five cases were audited:

- (A1) **Baseline:** static routing, no refinement.
- (A2) **Space bits only:** Router-side direction refinement, no processor-mask reduction.
- (A3) **Mask only:** processor-mask refinement only, no direction refinement.
- (A4) **Both:** direction refinement and processor-mask refinement together.
- (A5) **Null dispatch:** a null-compatible dispatch class intended to recover standard behavior exactly.

5 Benchmark Audit Results

Table 2 summarizes the audited results. Values are reported as mean $\pm$ population standard deviation across 960 comparisons for each ablation.

The combined method achieved a mean reduction of 18.28% in the synthetic energy proxy, with a standard deviation of 12.76 percentage points and a minimum observed regression of approximately -12.0% . This makes the central result more nuanced than a single headline figure: the method helps on average in the present software audit, but the effect is variable and not uniformly positive.

The ablation study identifies the main source of the apparent gain. **Processor-mask refinement dominates.** The mask-only condition produced a mean energy-proxy reduction of 16.88%, while space bits alone produced only 1.40% on average and occasionally harmed performance. This

Table 2: Audit summary over repeated virtual-mode experiments. Positive percentages indicate improvement relative to the static baseline. The null dispatch class recovers the baseline exactly.

Ablation	Energy reduction (%)	Latency reduction (%)	Path-change fraction	Sample count
Baseline	0.00 ± 0.00	0.00 ± 0.00	0.00	960
Null dispatch	0.00 ± 0.00	0.00 ± 0.00	0.00	960
Space bits only	1.40 ± 4.08	3.81 ± 10.02	0.15	960
Mask only	16.88 ± 12.41	0.00 ± 0.00	0.00	960
Both	18.28 ± 12.76	3.81 ± 10.02	0.15	960

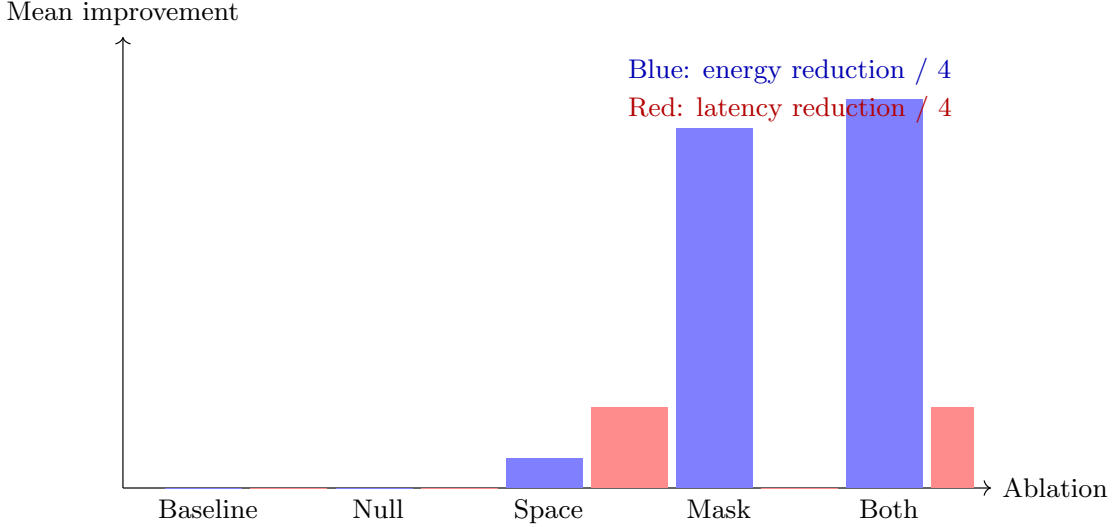


Figure 2: Audited ablation results. Processor-mask refinement accounts for most of the energy-proxy reduction, while space-bit direction refinement alone has a smaller and less stable effect. Values are scaled for compact plotting.

suggests that, in the present software model, local-core fan-out reduction explains most of the energy-proxy effect. Space-bit direction refinement contributes comparatively little on its own and is less stable.

The null-dispatch case is an important sanity check. It exactly recovers baseline behavior in both the energy and latency proxies, which supports the paper’s backward-compatibility claim in the present benchmark model.

5.1 Stress-test interpretation

The combined method helped more in some regimes than others. It performed better for clustered destinations than for dispersed ones, slightly better under high routing pressure than under low pressure, and somewhat better on the smaller 8×8 topology than on the larger 12×12 topology. These trends are consistent with the intuition that topology-aware refinement is most useful when traffic retains spatial structure. However, the variance is large enough that these trends should be treated as evidence of sensitivity rather than as universal law.

5.2 Conservative interpretation

In this audited virtual-mode benchmark, the combined refinement reduced a synthetic energy proxy on average, but the magnitude of the reduction depended strongly on the modeling assumptions and workload structure. Processor-mask refinement appears to be the dominant contributor in the current model. Space-bit direction refinement alone is smaller, less stable, and can harm performance in some cases. These are software-audit results. Whether they translate to real hardware energy savings remains unknown.

6 Limitations

The present evaluation has several important limitations.

- (L1) The audit is a virtual-mode software benchmark rather than a physical SpiNNaker board study.
- (L2) The energy and latency quantities are synthetic proxies, not hardware-calibrated measurements.
- (L3) The routing model is intentionally simplified and should not be interpreted as a full bit-accurate reproduction of production SpiNNaker routing microarchitecture. In particular, the benchmark models a four-direction rectangular torus rather than the six-direction hexagonal mesh of physical SpiNNaker hardware, which affects both routing path options and congestion behavior.
- (L4) The observed gains depend materially on a proxy model that rewards smaller processor masks by construction; this makes the results suitable for comparative hypothesis testing, but not for strong hardware-energy claims.
- (L5) The workloads are synthetic stress cases rather than full application traces, so transfer to deployed workloads remains uncertain.

These limitations do not negate the audit; they define its legitimate interpretive range. The results should be treated as reproducible software evidence about sensitivity and plausibility, not as settled hardware evidence.

7 Reproducibility and Code Availability

To support independent replication, the benchmark code used in this study is organized as a standalone experiment harness. The main audit driver is `benchmark_audit.py`, which writes raw trial data, aggregated JSON summaries, and LaTeX-ready table rows. The reproducibility package accompanying this work includes the routing prototypes, the benchmark scripts, the raw outputs, the environment metadata, and the exact run commands used in the audit.

The complete implementation, benchmark outputs, and reproducibility materials for this study are provided in the BGeckoResearch GitHub repository:

<https://github.com/BGeckoResearch/topology-aware-spinnaker>.

Only representative excerpts are included in the manuscript itself so that the paper remains readable while still being falsifiable and reproducible.

8 Conclusion

Topology-aware packet classification and adaptive local-core selection are worth pursuing because routing efficiency in SpiNNaker depends directly on spatial traffic structure, and the dispatch-class approach targets that dependency explicitly. This paper establishes the proposal on both architectural and experimental grounds. In the audited virtual-mode benchmark, the combined refinement reduced a synthetic energy proxy by 18.28% on average relative to the static baseline, but with substantial variance and occasional regressions.

The audit also clarifies the mechanism. Processor-mask refinement is the dominant source of the observed proxy reduction in the present model, while space-bit direction refinement alone has smaller and less stable effects. That is a useful result because it helps focus future work on the most consequential part of the proposal. The next step is to tighten the routing model further, evaluate richer workloads, and ultimately test the idea against hardware-aware measurements.

A Appendix: Representative Implementation Snippets

This appendix includes short representative listings only. The complete implementation is provided in the external reproducibility archive referenced above.

A.1 Core proxy definition

```
def energy_proxy(hops, mask_size, congested_hops):
    return hops * 1.0 + 0.18 * mask_size + 0.9 * congested_hops

def latency_proxy(hops, congested_hops):
    return hops * 1.0 + 1.75 * congested_hops
```

A.2 Representative routing/refinement snippet

```
def tested_policy(ablation, source, destination, width, height,
                  hotspots, mask_target, routing_pressure):
    base_order = preferred_directions(source, destination, width, height)
    blocked = hotspots if routing_pressure == "high" else \
        set(list(hotspots)[: max(1, len(hotspots)//2)])

    if ablation in {"baseline", "null_dispatch"}:
        return bfs_path(source, destination, width, height,
                        base_order, set()), 18
    if ablation == "space_bits_only":
        restricted = [d for d in base_order if d in base_order[:2]] + \
            [d for d in base_order if d not in base_order[:2]]
        path = bfs_path(source, destination, width, height,
                        restricted, blocked)
        if not path:
            path = bfs_path(source, destination, width, height,
                            base_order, set())
        return path, 18
    if ablation == "mask_only":
```



```
    return bfs_path(source, destination, width, height,
                    base_order, set()), mask_target
```

A.3 Representative benchmark driver snippet

```
for width, height in TOPOLOGIES:
    _ = virtual_machine(width, height)
    for seed in SEEDS:
        rng = random.Random(seed + width * 100 + height)
        for traffic_level in TRAFFIC_LEVELS:
            for dest_pattern in DEST_PATTERNS:
                for mask_target in MASK_SIZES:
                    for pressure in ROUTING_PRESSURE:
                        source, destination, hotspots = make_workload(
                            rng, width, height, traffic_level, dest_pattern)
```